参赛编号：	DAC23002052
赛题题号：	A

标题　电商平台用户行为分析与挖掘

摘　要

随着电子商务的迅猛发展，对电商平台用户行为的深入分析变得至关重要。本文以“电商平台用户行为分析与挖掘”为研究主题，旨在通过数据分析方法深入理解用户行为，以指导电商平台的营销策略和用户体验优化。研究分为五个主要部分，涉及从数据处理到模型构建和策略制定的全过程。

首先，我们对电商平台的详细交易数据进行了统计分析，包括用户的购买频率、交易金额、优惠券使用情况等。通过数据可视化技术，我们揭示了消费行为的时间分布和地理分布，识别了关键的购买趋势和消费模式。其次，文章利用特征工程技术，从原始交易数据中提取了关键特征，包括用户特征、商户特征和优惠券使用情况。这些特征为后续的机器学习模型提供了强有力的支持，以准确预测用户行为。在用户画像构建部分，我们运用了多维度数据分析，结合用户的个人属性、消费偏好和行为模式，创建了细致的用户画像。这些画像不仅帮助我们更好地理解用户群体，还为个性化营销提供了依据。针对电商平台的代金券发放策略，本文构建了一个预测模型，准确地预测了哪些用户更可能使用代金券。模型的构建基于用户过往的交易历史和行为模式，通过算法优化，实现了较高的预测准确率。最后，结合模型分析结果，本文设计了一套创新的优惠券投放策略。这些策略旨在提高优惠券的使用率和用户满意度，同时为电商平台带来更高的回报率。

本文不仅提供了一套全面的方法来分析和理解电商平台用户行为，还提出了实际可行的策略，以提升电商平台的用户体验和营销效果。

关键词：数据分析、特征提取、机器学习、预测

目　录

摘　要 1
背景 1
任务一 2
1.1 解决思路 2
1.2 数据预处理 3
1.2.1 读取数据 3
1.2.2 数据预处理 3
1.3 用户维度 4
1.3.1 用户消费总额统计 4
1.3.2 用户的平均订单金额 4
1.3.3 各省份、城市的消费金额、消费数量、用户数量占比 5
1.4 订单维度 7
1.4.1 订单购买数量分布 7
1.4.2 实付金额分布 7
1.4.3 邮费分布 8
1.4.4 优惠券使用率 9
1.4.5 订单销量随时间的变化 10
1.5 商家维度 11
任务二 13
2.1 解决思路 13
2.2 有效特征 13
任务三 17
3.1 解决思路 17
3.2 问题解决 17
任务四 20
4.1 解决思路 20
4.2 解决方法 20
4.3 模型评估 23
任务五 29
5.1 解决思路 29
5.2 解决方法 29
参考文献 30
附录 31

背景

电商是当今用户最大的交易市场之一，电商行业也逐渐成熟，所有市场中可售卖的商品全都在平台中存在，并且在网络和疫情的影响下，在线上的消费行为满足全年龄段用户。 用户的交易行为推动了电商的发展。对于电商平台来说，通过用户的交易行为、交易金额、交易频率、交易时间等信息，分析用户的行为，可促进二次营销，精准营销，促活等。

任务描述

1.对电商平台用户的消费行为进行统计分析并可视化。

2. 有这么一句话在业界广泛流传：“数据和特征决定了机器学习的上限，而模型和算法只是逼近这个上限而已”。特征工程是指利用领域知识从原始数据中提取有效特征，从而达到提高机器学习过程的结果的质量的目的。请根据用户的交易行为数据构造用户、商户和优惠券等有效特征。

3.用户画像是指根据用户的属性、用户偏好、用户行为等信息而抽象出来的标签化用户模型，通过给用户贴标签的方式来高度概括和描述用户，可以让人更直观地了解用户特征。请构建模型进行客户价值分析，并绘制用户画像分析用户特点。

4.平台推出不同商家的代金券，用于发放给全量用户。请根据用户的交易行为构建模型并训练，预测是否为该用户发放商家代金券，最后评估模型性能。

5.请设计一份优惠券投放策略。

任务一

1.1 解决思路

通过对提供的数据进行预处理、分析；数据集包含以下列：

- 订单 ID：订单的唯一标识；
- 商家 ID：商家的唯一标识；
- 用户 ID：用户的唯一标识；
- 付款日期：付款发生的日期和时间；
- 使用状态：用户对优惠券的使用状态（1：已使用，0：领取未使用，None：未领取）；
- 实付金额：用户实际支付的金额，负数表示未使用优惠券；
- 邮费：订单的邮费；
- 省份、城市：用户的地理位置信息；
- 购买数量：购买的商品数量；

针对电商平台用户的消费行为进行统计分析，可以分为三个维度：用户维度、商家维度、订单维度。

用户维度下，统计各用户的消费金额总额、每笔订单平均金额的分布情况，用户地址分布情况、各地址下的消费金额、消费数量分布情况。

订单维度下，每笔订单的购买数量分布、实付金额分布、邮费分布（包邮比例）、优惠券使用率。

商家维度下，各商家商品总销售量、销售金额的分布情况；优惠券，优惠券使用状态分布、使用率，不同商家、不同商品下优惠券的使用率。

1.2 数据预处理

1.2.1 读取数据

读取保存在本地的 data.xlsx 文件并检查其结构，如代码 1 所示：

代码 1 读取文件

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# 读取数据
file_path = './data/data.xlsx'
data = pd.read_excel(file_path)
print(data[:10])
```

数据集的基本结构和统计信息如下：

- 数据集包含 20183 行和 10 列；
- 大多数列没有缺失值，但‘使用状态’列有 354 个缺失值；
- 实付金额列的值范围从-300 到 1028，正值表示使用了优惠券，负值表示付款时未使用或未领取的优惠券；

1.2.2 数据预处理

1. 处理缺失值：‘使用状态’列中的缺失值被解释为优惠券未领取，因此可以将缺失值替换为 2，表示未领取；

2. 数据类型转换：将付款日期列转换为能反映年、月、日、小时等的多个特征；

3. 异常值处理：对于‘实付金额’中存在的负数(代表付款时没有使用优惠卷，但是付款金额是绝对值)转变为绝对值

4. 分类数据编码：对于‘省份’，‘城市’等分类数据，使用标签编码(Label-encoding)

具体如代码 2 所示：

代码 2 数据预处理

```python
data['使用状态'].fillna(2, inplace=True)

data['付款日期'] = pd.to_datetime(data['付款日期'])
data['年月'] = data['付款日期'].dt.to_period('M')

data['实付金额'] = data['实付金额'].abs()

label_encoder = LabelEncoder()
for column in ['省份', '城市']:
    data[column] = label_encoder.fit_transform(data[column])
```

1.3 用户维度

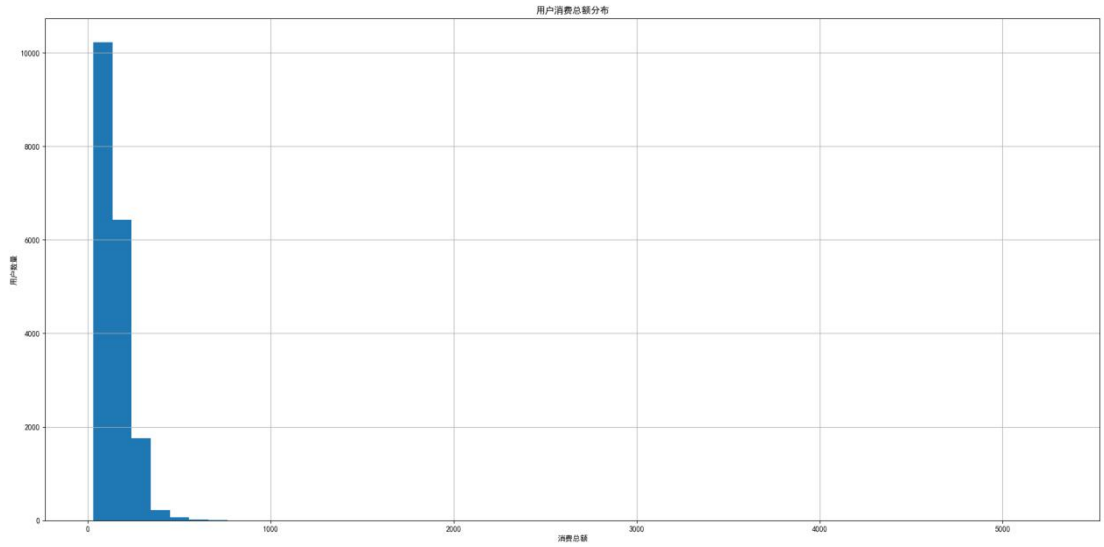
1.3.1 用户消费总额统计

将数据 data 按照‘用户 ID’列的不同值进行分组，[’实付金额’]指定了要计算分组后的数据中感兴趣的列，对感兴趣列中值进行累加，计算出用户的总消费金额并进行可视化。具体如代码 3 所示：

代码 3 用户消费总额统计

```
# 计算每个用户的消费总额
user_total_spending = data.groupby('用户ID')['实付金额'].sum()
plt.figure(figsize=(10, 6))
user_total_spending.hist(bins=50)
plt.title('用户消费总额分布')
plt.xlabel('消费总额')
plt.ylabel('用户数量')
plt.show()
```

结果 1 用户消费总额柱状图



从结果 1 中可以看出，大部分用户的消费总额较低，随着消费总额的增加，达到该消费金额的用户数量迅速减少，表明有大量用户进行小额消费，而大额消费的用户相对较少。

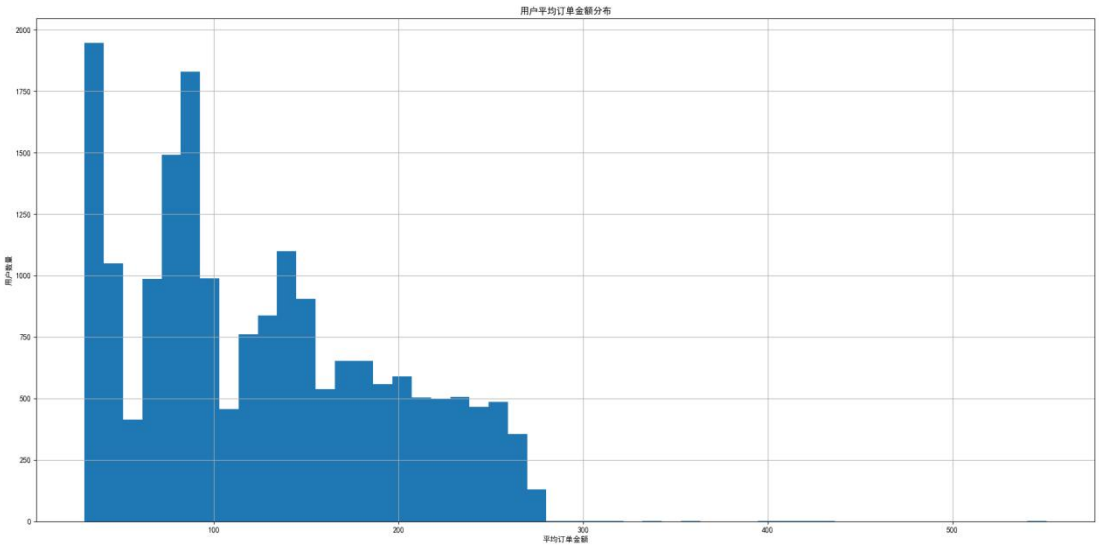
1.3.2 用户的平均订单金额

结果 1 已经计算出用户消费总额，然后使用同样的方法对用户的平均订单金额进行统计；实现如代码 4 所示。

代码 4 用户的平均订单金额

```
# 计算每个用户的平均订单金额
average_order_amount = data.groupby('用户ID')['实付金额'].mean()
plt.figure(figsize=(10, 6))
average_order_amount.hist(bins=50)
plt.title('用户平均订单金额分布')
plt.xlabel('平均订单金额')
plt.ylabel('用户数量')
plt.show()
```

结果 2 用户的平均订单金额柱状图



从结果 2 中可以看出，用户的平均订单金额主要集中在较低的金额区间，多数用户的平均购买金额不高，随着平均订单金额的增加，购买此金额区间的用户数量逐渐减少。

1.3.3 各省份、城市的消费金额、消费数量、用户数量占比

将相同省份且城市的数据行聚集在一起，对每个省份和城市组合的‘实付金额’进行求和，得到每个省份和城市组合的总消费金额；计算每个省份和城市组合中的数据行数，得到每个省份和城市组合的订单数量；对‘用户 ID’进行去重之后计算每个省份出现的次数，即每个省份的用户数量，。实现如代码 5 所示。

代码 5 各省份、城市的消费金额、消费数量、用户数量占比

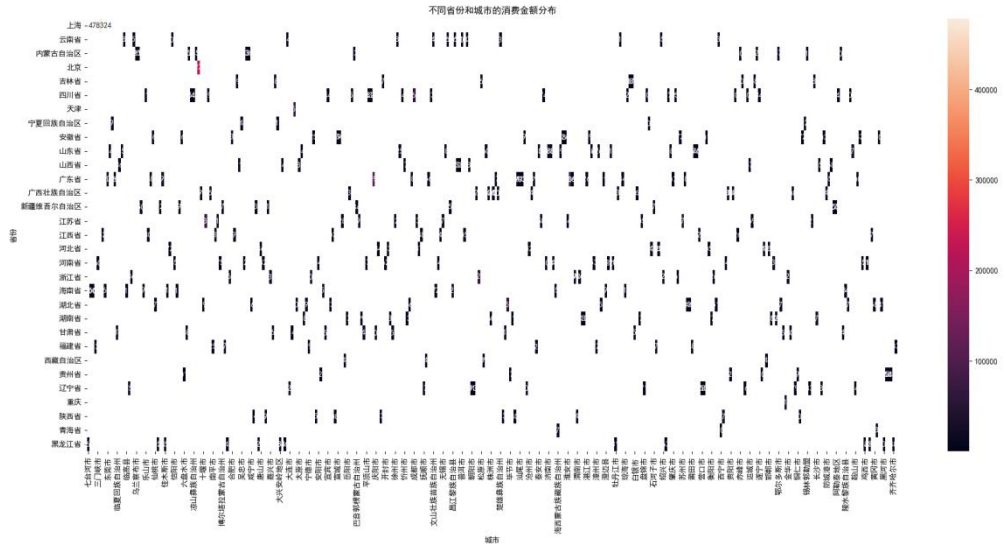
```
# 各省份、城市的消费金额和消费数量
province_spending = data.groupby('省份')['实付金额'].sum()
city_spending = data.groupby(['省份', '城市'])['实付金额'].sum()
province_order_count = data.groupby('省份').size()
city_order_count = data.groupby(['省份', '城市']).size()


province_city_spending = data.pivot_table(index='省份', columns='城市', values='实付金额', aggfunc='sum')
plt.figure(figsize=(20, 10))
sns.heatmap(province_city_spending, annot=True, fmt='.0f')
plt.title('不同省份和城市的消费金额分布')
plt.xlabel('城市')
plt.ylabel('省份')
plt.show()

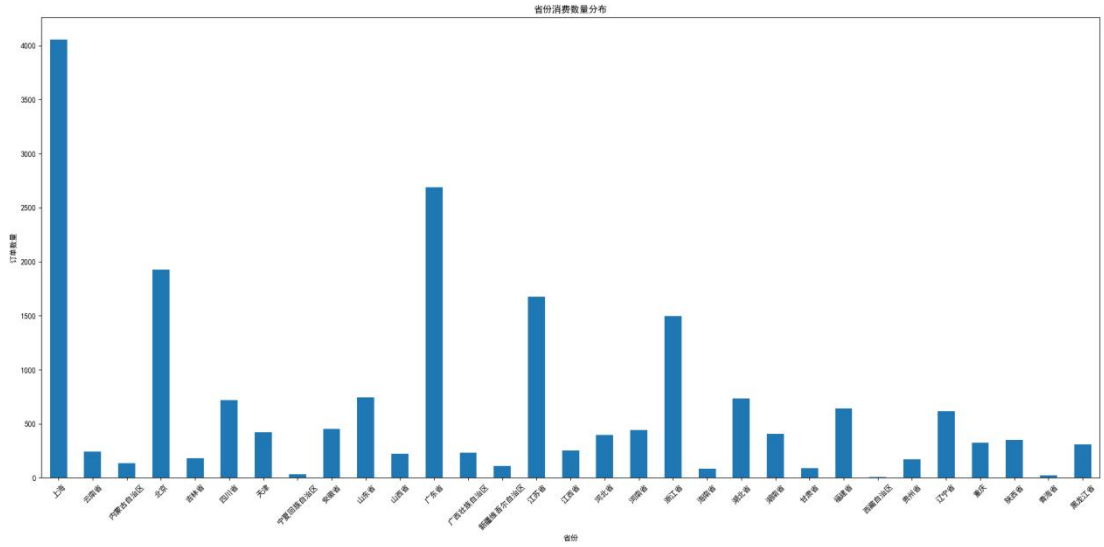
plt.figure(figsize=(20, 10), dpi=100)
province_order_count.plot(kind='bar')
plt.title('省份消费数量分布')
plt.xlabel('省份')
plt.ylabel('订单数量')
plt.xticks(rotation=45)
plt.show()
```

```
province_counts = data.drop_duplicates(subset=['用户ID', '省份'])['省份'].value_counts()
plt.figure(figsize=(10, 6))
province_counts.plot(kind='pie', autopct='%1.1f%%')
plt.title('不同省份的用户数量占比')
plt.ylabel('')  # 隐藏y轴标签
plt.show()
```

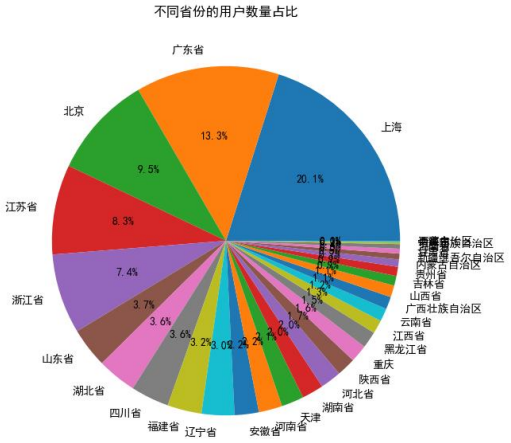
结果 3 各省份、城市的消费金额柱状图



结果 4 各省份、城市的消费数量柱状图



结果 5 不同省份的用户数量占比



注：由于在预处理部位‘省份’和‘城市’已经采取了标签编码(Label-encoding)，结果 3 和结果 4 中的省份应该是数字来代表，但是为了更方便的可视化，在结果 3、结果 4 和结果 5 中暂时取消了标签编码

从上面的结果可以得出用户数量、消费金额、消费数量和城市的发达程度是成正比的。

1.4 订单维度

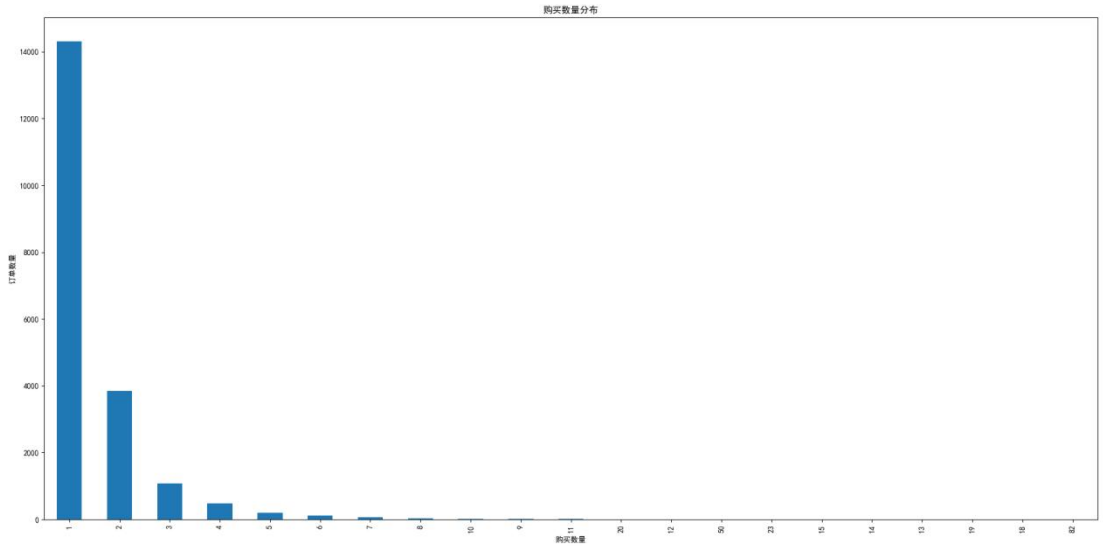
1.4.1 订单购买数量分布

通过计算‘购买数量’列中每个不同的购买数量值的出现次数，统计订单购买数量的分布情况并进行可视化。具体实现如代码 6 所示。

代码 6 购买数量分布

```
# 每笔订单的购买数量分布
order_quantity_distribution = data['购买数量'].value_counts()
plt.figure(figsize=(20, 10))
order_quantity_distribution.plot(kind='bar')
plt.title('购买数量分布')
plt.xlabel('购买数量')
plt.ylabel('订单数量')
plt.show()
```

结果 6 订单购买数量分布柱状图



由结果 6 我们可以得出的结论：随着购买数量的增加，订单数量减少，二者成反比；绝大多数用户的购买数量集中在 1 和 2。

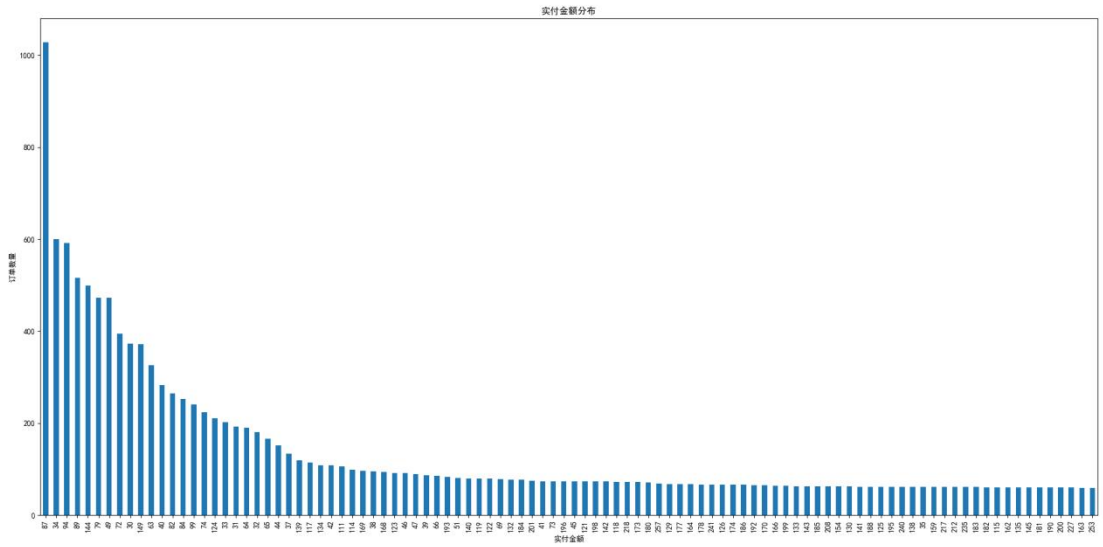
1.4.2 实付金额分布

因为‘实付金额’列中的金额列别无法全部进行可视化，所以读取‘实付金额’前 100 个值进行可视化。具体实现如代码 7 所示。

代码 7 实付金额分布

```
# 实付金额分布
payment_distribution = data['实付金额'].value_counts().nlargest(100)
plt.figure(figsize=(20, 10))
payment_distribution.plot(kind='bar')
plt.title('实付金额分布')
plt.xlabel('实付金额')
plt.ylabel('订单数量')
plt.show()
```

结果 7 实付金额分布柱状图



结果 7 表现实付金额为 87 的订单出现的次数最多，代表了该金额点的商品可能特别受欢迎或者常见，更加容易被用户接受。

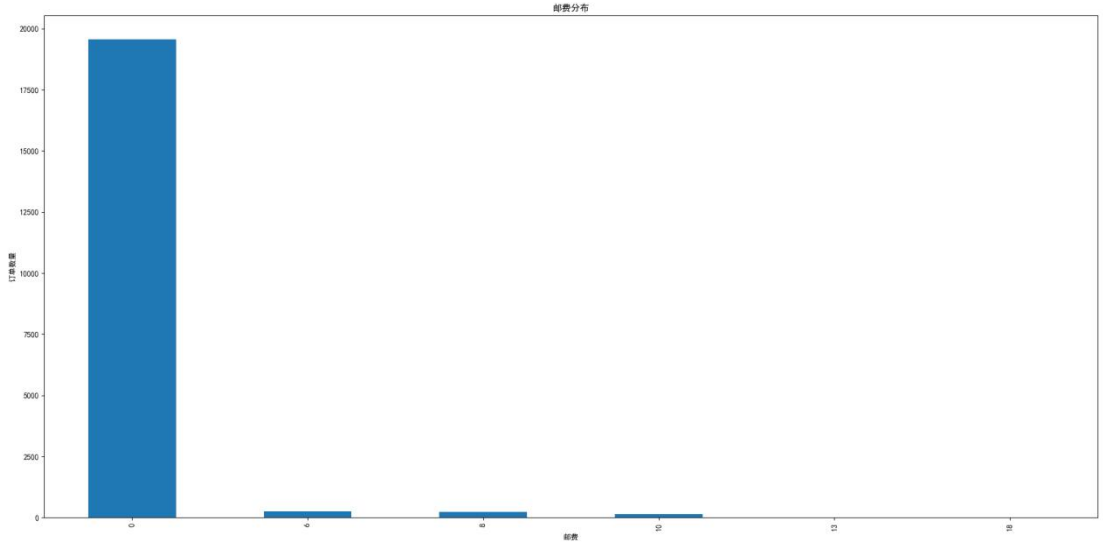
1.4.3 邮费分布

首先计算了免邮费订单的比例，检查订单是否具有 0 邮费的结果。如果订单具有 0 邮费，则为 True，否则为 False；然后通过‘邮费’列的全部值计算不同邮费金额的数量，返回一个包含邮费值和对应订单数量并可视化不同邮费金额的订单数量分布情况。具体代码实现如代码 8 所示。

代码 8 邮费分布

```
# 邮费分布（包邮比例）
free_shipping_ratio = (data['邮费'] == 0).mean()
plt.figure(figsize=(20, 10))
data['邮费'].value_counts().plot(kind='bar')
plt.title('邮费分布')
plt.xlabel('邮费')
plt.ylabel('订单数量')
plt.show()
```

结果 8 邮费分布柱状图



从结果 8 中得出结论：免费邮费的比例占据了绝大部分，用户更偏向于享受免费邮费商品。

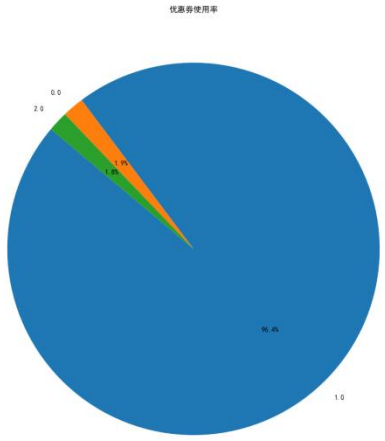
1.4.4 优惠券使用率

计算优惠券的使用率，并生成一个饼图，以可视化不同优惠券使用状态的比例。饼图展示了使用了优惠券的订单与未使用优惠券的订单之间的比例关系；具体实现如代码 9 所示。

代码 9 优惠券使用率

```
59  # 优惠券使用率
60  coupon_usage_rate = (data['使用状态'] == 1).mean()
61  coupon_usage = data['使用状态'].value_counts()
62  plt.figure(figsize=(10, 10))
63  coupon_usage.plot(kind='pie', autopct='%1.1f%%', startangle=140)
64  plt.title('优惠券使用率')
65  plt.ylabel('')
66  plt.show()
```

结果 9 优惠券使用率饼图



由结果 9 得知：使用了优惠券的用户占比了 96.4%，绝大多数用户偏向于使用优惠券

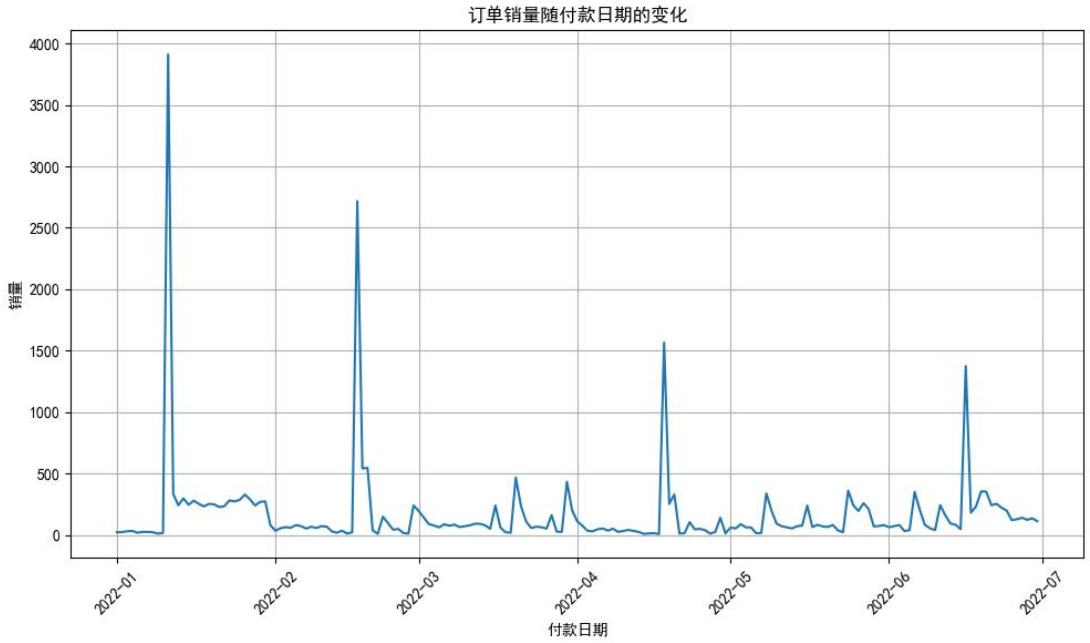
1.4.5 订单销量随时间的变化

将订单销量随着付款日期的变化绘制成折线图，以便观察销量的趋势和波动。

代码 10 订单销量随时间的变化

```
data['付款日期'] = pd.to_datetime(data['付款日期']).dt.date
sales_data = data.groupby('付款日期')['购买数量'].sum().reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_data['付款日期'], sales_data['购买数量'])
plt.title('订单销量随付款日期的变化')
plt.xlabel('付款日期')
plt.ylabel('销量')
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```

结果 10 订单销量随时间的变化折线图



由结果 10 可以总结出：在每个月的中旬订单销量就会上涨，可能和用户的工资发放时间有关；而在每个月开头和结尾订单量就会减少。

1.5 商家维度

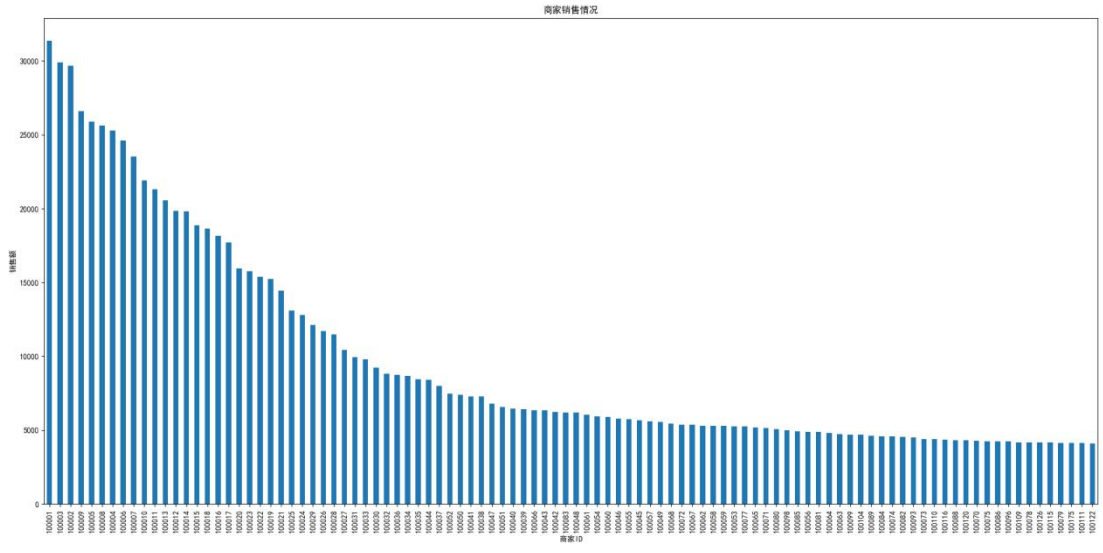
考虑无法将所有的商家 ID 都进行可视化，所以文中只对前 100 个商家的销售情况和订单数量进行了可视化。具体实现如代码 11 所示。

代码 11 前 100 商家销售情况、订单数量

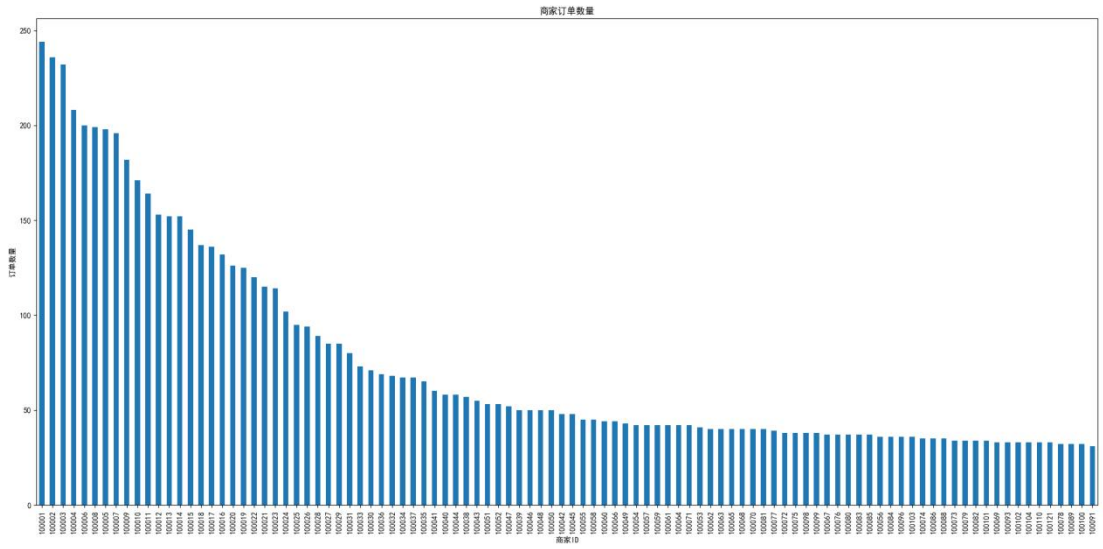
```
# 各商家销售情况
merchant_sales = data.groupby('商家ID')['实付金额'].sum().nlargest(100)
merchant_order_count = data.groupby('商家ID').size().nlargest(100)
plt.figure(figsize=(20, 10))
merchant_sales.plot(kind='bar')
plt.title('商家销售情况')
plt.xlabel('商家ID')
plt.ylabel('销售额')
plt.xticks(rotation=90)
plt.show()

# 商家订单数量
plt.figure(figsize=(20, 10))
merchant_order_count.plot(kind='bar')
plt.title('商家订单数量')
plt.xlabel('商家ID')
plt.ylabel('订单数量')
plt.xticks(rotation=90)
plt.show()
```

结果 11 商家销售情况



结果 12 商家订单数量



可以看出来商家销售情况、订单数量和商家的排名成正比；商家排名越靠前，销售情况就会越好

任务一总结：

包邮订单的比例约为 96.90%。优惠券使用率为 96.4%（已使用优惠券），3.6%（未使用优惠券）；通过对用户维度、商家维度、订单维度下进行统计分析，可以得到结论：

绝大对数用户在领取到优惠券之后会使用优惠券且更偏向于免费邮费。

任务二

2.1 解决思路

建模思路：根据订单维度建模，对每一笔订单而言，如果使用了优惠券、那么这个用户就是需要优惠券的；所以本文在基准订单/基准商家下，按照历史该用户订单数据来构建特征，统计历史订单数据。省份和城市作为单独变量进行考虑。

2.2 有效特征

包括以下有效特征：

对于用户维度：

- 订单数：用户的订单总数；
- 总金额：用户支付的总金额；
- 平均订单金额：用户支付的平均订单金额；
- 购买商品总数：用户购买的商品总数量；
- 购买商品平均数：用户平均每次购买的商品数量；
- 邮费金额总数：用户支付的邮费总金额；
- 邮费金额平均数：用户支付的平均邮费金额；
- 包邮订单数：用户的包邮订单总数；
- 优惠券使用数量：用户使用优惠券的次数；
- 优惠券使用率：用户使用优惠券的比率；
- 邮费占总金额比率：邮费占用户支付总金额的比率；

对于商户维度：

- 订单数：商户的订单总数；
- 总金额：商户销售的总金额；
- 平均订单金额：商户销售的平均订单金额；
- 售出商品总数：商户售出的商品总数量；
- 售出商品平均数：商户平均每笔订单售出的商品数量；
- 邮费金额总数：商户收取的邮费总金额；
- 邮费金额平均数：商户收取的平均邮费金额；
- 包邮订单数：商户的包邮订单总数；

- 优惠券使用数量：通过商户使用优惠券的次数；
- 优惠券使用率：通过商户使用优惠券的比率；
- 邮费占总金额比率：邮费占商户销售总金额的比率；

对于优惠券维度：

- 优惠券使用率：并和上面两个特征进行了合并特征

使用 pandas 库读取 data.xlsx 数据，将其转化为 DataFrame 类型数据，由于用户 ID 和商家 ID 是具有唯一性的,所以本文以这两个字段为依据进行统计，对数据进行分组提取有效特征。具体实现如代码 12 所示。

代码 12 用户特征、商家特征、优惠券特征

```
# 用户特征
user_features = data.groupby('用户ID').agg({
    '订单ID': 'nunique',
    '实付金额': ['sum', 'mean'],
    '购买数量': ['sum', 'mean'],
    '邮费': ['sum', 'mean', lambda x: (x == 0).sum()]
})

user_features.columns = ['订单数', '总金额', '平均订单金额', '购买商品总数', '购买商品平均数', '邮费金额总数', '邮费金额平均数', '包邮订单数']
user_features['优惠券使用数量'] = data.groupby('用户ID')['使用状态'].apply(lambda x: (x == 1).sum())
user_features['优惠券使用率'] = data.groupby('用户ID')['使用状态'].apply(lambda x: (x == 1).sum() / len(x))
user_features['邮费占总金额比率'] = user_features['邮费金额总数'] / user_features['总金额']

# 商户特征
merchant_features = data.groupby('商家ID').agg({
    '订单ID': 'nunique',
    '实付金额': ['sum', 'mean'],
    '购买数量': ['sum', 'mean'],
    '邮费': ['sum', 'mean', lambda x: (x == 0).sum()]
})
merchant_features.columns = ['订单数', '总金额', '平均订单金额', '售出商品总数', '售出商品平均数', '邮费金额总数', '邮费金额平均数', '包邮订单数']
merchant_features['优惠券使用数量'] = data.groupby('商家ID')['使用状态'].apply(lambda x: (x == 1).sum())
merchant_features['优惠券使用率'] = data.groupby('商家ID')['使用状态'].apply(lambda x: (x == 1).sum() / len(x))
merchant_features['邮费占总金额比率'] = merchant_features['邮费金额总数'] / merchant_features['总金额']

# 优惠券特征
coupon_usage = data.groupby(['用户ID', '使用状态']).size().unstack(fill_value=0)
coupon_usage['优惠券使用率'] = coupon_usage['已使用'] / (coupon_usage['已使用'] + coupon_usage['领取未使用'] + coupon_usage['未领取'])
```

结果 13 用户特征可视化

用户ID	订单数	总金额	平均订单金额	购买商品总数	购买商品平均数	邮费金额总数	邮费金额平均数	包邮订单数	优惠券使用数量	优惠券使用率	邮费占总金额比率
姐美丽张8	1	177	177.0	1	1.0	8	8.0	0	1	1.0	0.045198
琪朴诺	1	94	94.0	1	1.0	0	0.0	1	1	1.0	0.000000
7明阳6	1	151	151.0	1	1.0	0	0.0	1	1	1.0	0.000000
0seea10	1	82	82.0	6	6.0	0	0.0	1	1	1.0	0.000000
8tb07026168	1	79	79.0	1	1.0	0	0.0	1	1	1.0	0.000000
2gabriel35	2	236	118.0	3	1.5	0	0.0	2	2	1.0	0.000000
0菜园小虫子24	1	126	126.0	2	2.0	0	0.0	1	1	1.0	0.000000
宁chri宁	1	138	138.0	2	2.0	0	0.0	1	1	1.0	0.000000
c梅m	1	270	270.0	1	1.0	0	0.0	1	1	1.0	0.000000
2lucifer01	2	262	131.0	4	2.0	0	0.0	2	2	1.0	0.000000
econtend	1	203	203.0	1	1.0	0	0.0	1	1	1.0	0.000000
2htc8	1	109	109.0	1	1.0	0	0.0	1	1	1.0	0.000000
5yufangh97	1	200	200.0	1	1.0	0	0.0	1	1	1.0	0.000000
4zhoulu3741386	1	214	214.0	1	1.0	0	0.0	1	1	1.0	0.000000
7桃粑7	1	65	65.0	1	1.0	0	0.0	1	0	0.0	0.000000
6风轻眠36	1	193	193.0	1	1.0	0	0.0	1	1	1.0	0.000000
lpurplela	1	79	79.0	1	1.0	0	0.0	1	1	1.0	0.000000
n豆豆宝宝yg	1	89	89.0	1	1.0	0	0.0	1	1	1.0	0.000000
ihoumina	1	49	49.0	1	1.0	0	0.0	1	1	1.0	0.000000
以聆	1	228	228.0	1	1.0	0	0.0	1	1	1.0	0.000000

注：由于用户 ID 过多，无法全部进行可视化，所以随机抽取了 20 个用户 ID 进行特征打印

结果 14 商家特征可视化

商家ID	订单数	总金额	平均订单金额	售出商品总数	售出商品平均数	邮费金额总数	邮费金额平均数	包邮订单数	优惠券使用数量	优惠券使用率	邮费占总金额比率
102137	1	143	143.000000	2	2.000000	0	0.000000	1	0	0.0	0.000000
100302	15	1493	99.533333	29	1.933333	10	0.666667	14	15	1.0	0.006698
103411	1	190	190.000000	1	1.000000	0	0.000000	1	1	1.0	0.000000
105637	1	149	149.000000	1	1.000000	0	0.000000	1	1	1.0	0.000000
100959	3	488	162.666667	3	1.000000	0	0.000000	3	3	1.0	0.000000
103488	1	144	144.000000	1	1.000000	0	0.000000	1	1	1.0	0.000000
102987	1	180	180.000000	1	1.000000	0	0.000000	1	1	1.0	0.000000
101955	2	293	146.500000	4	2.000000	0	0.000000	2	2	1.0	0.000000
102850	1	63	63.000000	3	3.000000	0	0.000000	1	1	1.0	0.000000
100925	2	283	141.500000	3	1.500000	0	0.000000	2	2	1.0	0.000000
103271	1	71	71.000000	3	3.000000	0	0.000000	1	1	1.0	0.000000
105128	1	30	30.000000	2	2.000000	0	0.000000	1	1	1.0	0.000000
102581	1	46	46.000000	4	4.000000	0	0.000000	1	1	1.0	0.000000
100348	8	806	100.750000	8	1.000000	0	0.000000	8	8	1.0	0.000000
102443	1	66	66.000000	1	1.000000	0	0.000000	1	1	1.0	0.000000
102867	1	147	147.000000	1	1.000000	0	0.000000	1	1	1.0	0.000000
103091	1	264	264.000000	1	1.000000	0	0.000000	1	1	1.0	0.000000
105337	1	146	146.000000	1	1.000000	0	0.000000	1	1	1.0	0.000000
102385	2	338	169.000000	3	1.500000	0	0.000000	2	2	1.0	0.000000
103079	1	94	94.000000	1	1.000000	0	0.000000	1	1	1.0	0.000000

注：由于商家 ID 过多，无法全部进行可视化，所以随机抽取了 20 个商家 ID 进行特征打印。

商家维度和时间窗口分析：

每个商家在不同时间窗口（年月）的订单数、总金额、平均订单金额、售出商品总数和平均数。具体实现如代码 13 所示：

代码 13 商家维度和时间窗口分析

```
data['付款日期'] = pd.to_datetime(data['付款日期'])
data['年月'] = data['付款日期'].dt.to_period('M')
merchant_time_analysis = data.groupby(['商家ID', '年月']).agg({
    '订单ID': 'nunique',
    '实付金额': ['sum', 'mean'],
    '购买数量': ['sum', 'mean']
}).reset_index()
merchant_time_analysis.columns = ['商家ID', '年月', '订单数', '总金额', '平均订单金额', '售出商品总数', '售出商品平均数']
print(merchant_time_analysis.head(20))
```

结果 14 商家维度和时间窗口分析可视化

	商家ID	年月	订单数	总金额	平均订单金额	售出商品总数	售出商品平均数
0	100001	2022-01	221	27212	123.131222	328	1.484163
1	100001	2022-02	10	1693	169.300000	12	1.200000
2	100001	2022-03	5	875	175.000000	8	1.600000
3	100001	2022-04	1	151	151.000000	2	2.000000
4	100001	2022-05	6	1268	211.333333	7	1.166667
5	100001	2022-06	1	155	155.000000	1	1.000000
6	100002	2022-01	193	24062	124.673575	245	1.269430
7	100002	2022-02	27	3209	118.851852	43	1.592593
8	100002	2022-03	6	1035	172.500000	6	1.000000
9	100002	2022-04	1	99	99.000000	2	2.000000
10	100002	2022-05	5	586	117.200000	13	2.600000
11	100002	2022-06	4	676	169.000000	4	1.000000
12	100003	2022-01	172	21035	122.296512	230	1.337209
13	100003	2022-02	31	4552	146.838710	37	1.193548
14	100003	2022-03	17	2700	158.823529	21	1.235294
15	100003	2022-04	5	621	124.200000	8	1.600000
16	100003	2022-05	3	500	166.666667	5	1.666667
17	100003	2022-06	4	478	119.500000	6	1.500000
18	100004	2022-01	131	14948	114.106870	193	1.473282
19	100004	2022-02	38	5256	138.315789	62	1.631579

例如，商家 ID 为 100001 的商家在 2022 年 1 月的订单数为 221，总金额为

24572 元，平均订单金额为 111.19 元，售出商品总数为 328 个，平均每笔订单售出 1.48 个商品。结果依次类推。

省份和城市维度分析：

每个省份和城市在不同时间窗口的订单数、总金额、平均订单金额、售出商品总数和平均数。

代码 14 省份和城市维度分析

```
province_city_analysis = data.groupby(['省份', '城市', '年月']).agg({
    '订单ID': 'nunique',
    '实付金额': ['sum', 'mean'],
    '购买数量': ['sum', 'mean']
}).reset_index()
province_city_analysis.columns = ['省份', '城市', '年月', '订单数', '总金额', '平均订单金额', '商品总数', '商品平均数']
print(province_city_analysis.head(20))
```

结果 15 省份和城市维度分析可视化

	省份	城市	年月	订单数	总金额	平均订单金额	商品总数	商品平均数
0	上海	上海市	2022-01	1352	149294	110.424556	2206	1.631657
1	上海	上海市	2022-02	617	75302	122.045381	1010	1.636953
2	上海	上海市	2022-03	450	58707	130.460000	875	1.944444
3	上海	上海市	2022-04	391	47958	122.654731	665	1.700767
4	上海	上海市	2022-05	471	56787	120.566879	823	1.747346
5	上海	上海市	2022-06	775	90276	116.485161	1275	1.645161
6	云南省	临沧市	2022-02	1	149	149.000000	1	1.000000
7	云南省	临沧市	2022-03	1	194	194.000000	1	1.000000
8	云南省	临沧市	2022-05	2	190	95.000000	4	2.000000
9	云南省	临沧市	2022-06	1	52	52.000000	5	5.000000
10	云南省	丽江市	2022-01	1	100	100.000000	8	8.000000
11	云南省	丽江市	2022-03	1	158	158.000000	1	1.000000
12	云南省	丽江市	2022-05	2	301	150.500000	3	1.500000
13	云南省	保山市	2022-01	1	256	256.000000	1	1.000000
14	云南省	保山市	2022-02	1	31	31.000000	1	1.000000
15	云南省	保山市	2022-06	3	513	171.000000	3	1.000000
16	云南省	大理白族自治州	2022-01	1	56	56.000000	4	4.000000
17	云南省	大理白族自治州	2022-02	4	349	87.250000	7	1.750000
18	云南省	大理白族自治州	2022-03	2	269	134.500000	4	2.000000
19	云南省	大理白族自治州	2022-04	3	275	91.666667	4	1.333333

例如，在上海市，2022 年 1 月的订单数为 1352，总金额为 140784 元，平均订单金额为 104.13 元，商品总数为 2206 个，平均每笔订单 1.63 个商品。结果依次类推。

任务三

3.1 解决思路

客户价值是针对电商平台而言的，即用户给电商平台/商家带来了多少收入、多少利润，以及未来能够带来多少收入和利润，这里不仅要考虑历史订单金额、还要考虑每个用户历史消费频率（客户粘性）。所以本文选择使用 RFM 模型进行分析。

RFM 模型是一种常用的客户价值分析方法，它包括以下三个维度：

- R（Recency）：最近一次消费。用户最后一次购买距今的时间，反映了用户的活跃程度；
- F（Frequency）：消费频率。用户在特定时间内的购买次数，反映了用户的忠诚度；
- M（Monetary）：消费金额。用户在特定时间内的消费总金额，反映了用户的购买能力；

RFM 指标：

- Recency：自客户距离上次购买的天数；
- Frequency：购买次数；
- Monetary：消费总额；

这些指标提供了每位用户的基本购买行为概览。接下来，将这些指标分为不同的等级（如高、中、低），以便更好地进行用户画像分析。分级通过分位数（四分位数）来实现。根据四分位数对 RFM 指标进行分级，并为每个用户分配一个 RFM 得分，能够更清楚地刻画不同类别的用户特征。

3.2 问题解决

1. 确定最晚的付款日期，查找数据集中所有购买记录里最近一次的购买日期，加上一天之后作为参考日期，用于计算每个客户的 Recency 值；

2. 按照‘用户 ID’列进行分组，对于每一个用户组：

   - 计算其最近一次付款日期与参考日期之间的天数差(代表 Recency)；
   - 计算每个用户的唯一订单数量(代表 Frequency)；
   - 计算每个用户的所有订单的总实付金额(Monetary)；

3. 制定 RFM 评分函数(rfm_score)根据四分位数给每个 RFM 指标打分。分值越低表示表现越好（例如，较近的购买日期得到较低的 Recency 分数)；

4. 对每个用户的 Recency、Frequency 和 Monetary 应用评分函数，得到 R_Score、F_Score 和 M_Score；

5. 根据 RFM 总分，将客户分为高价值(分数 7 以上)、中等价值(分数 6)和低价值(分 6 以下)三个群体；

每位用户分配了基于 RFM 指标的评分。这个评分体系如下：

- R_Score（最近一次购买的时间）：1 表示最近购买，4 表示购买时间最远；
- F_Score（购买频率）：1 表示购买次数最少，4 表示购买次数最多；
- M_Score（消费金额）：1 表示消费总额最低，4 表示消费总额最高；
- RFM_Segment：三个分数的组合，如“311”；
- RFM_Score：三个分数之和，分数越高，客户价值越高；

具体实现如代码 15 所示：

代码 15 RFM 分析

```
reference_date = data['付款日期'].max() + pd.Timedelta(days=1)
rfm = data.groupby('用户ID').agg({
    '付款日期': lambda x: (reference_date - x.max()).days,
    '订单ID': 'nunique',
    '实付金额': 'sum'
}).reset_index()

rfm.columns = ['用户ID', 'Recency', 'Frequency', 'Monetary']

# Ensuring that the RFM metrics are numeric
rfm['Recency'] = pd.to_numeric(rfm['Recency'], errors='coerce')
rfm['Frequency'] = pd.to_numeric(rfm['Frequency'], errors='coerce')
rfm['Monetary'] = pd.to_numeric(rfm['Monetary'], errors='coerce')
#

quantiles = rfm[['Recency', 'Frequency', 'Monetary']].quantile(q=[0.25, 0.5, 0.75])

def rfm_score(x, p, d):
    if x <= d[p][0.25]:
        return 1
    elif x <= d[p][0.50]:
        return 2
    elif x <= d[p][0.75]:
        return 3
    else:
        return 4

rfm['R_Score'] = rfm['Recency'].apply(rfm_score, args=('Recency', quantiles))
rfm['F_Score'] = rfm['Frequency'].apply(rfm_score, args=('Frequency', quantiles))
rfm['M_Score'] = rfm['Monetary'].apply(rfm_score, args=('Monetary', quantiles))
rfm['RFM_Segment'] = rfm['R_Score'].map(str) + rfm['F_Score'].map(str) + rfm['M_Score'].map(str)
rfm['RFM_Score'] = rfm[['R_Score', 'F_Score', 'M_Score']].sum(axis=1)

# 识别高价值和低价值客户
high_value_customers = rfm[rfm['RFM_Score'] >= rfm['RFM_Score'].quantile(0.75)]
mid_value_customers = rfm[(rfm['RFM_Score'] > rfm['RFM_Score'].quantile(0.25)) & (rfm['RFM_Score'] < rfm['RFM_Score'].quantile(0.75))]
low_value_customers = rfm[rfm['RFM_Score'] <= rfm['RFM_Score'].quantile(0.25)]
print(high_value_customers.head())
print(mid_value_customers.head())
print(low_value_customers.head())
```

结果 16 RFM 分析可视化

高价值用户：

	用户ID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Segment	RFM_Score
1	.christiny	153	1	183	3	1	3	313	7
5	000米粒儿米粒0	149	1	170	3	1	3	313	7
6	00556旭79618	158	1	159	4	1	3	413	8
8	00青0	160	2	321	4	4	4	444	12
9	02000小轩3	134	1	230	3	1	4	314	8

潜在发展用户：

	用户ID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Segment	RFM_Score
2	.托托m	171	1	37	4	1	1	411	6
7	00不哭0	160	1	53	4	1	1	411	6
11	0839576292彩0	80	1	188	2	1	3	213	6
14	0abao1000	7	1	265	1	1	4	114	6
17	0adachen9	156	1	111	3	1	2	312	6

需关注用户：

	用户ID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Segment	RFM_Score
0	.blue_ram	147	1	49	3	1	1	311	5
3	0000妮	3	1	164	1	1	3	113	5
4	000xyx0	74	1	99	2	1	2	212	5
10	055等0	73	1	123	2	1	2	212	5
13	0aax06	8	1	89	1	1	2	112	4

基于这些分数进行用户画像的描述：

高价值用户（RFM_Score 高）：这些用户最近购买过，购买频率高，消费金额也高。他们可能是忠实客户，对品牌有高度忠诚度。

潜在发展用户（高 R_Score 和 M_Score，低 F_Score）：这些用户最近购买过，消费金额高，但购买频率较低。他们可能是新客户，有潜力成为忠实客户。

需关注用户（低 R_Score 和 F_Score）：这些用户购买频率低，且已经有一段时间没有购买。可能需要通过营销活动来重新吸引他们。

任务四

4.1 解决思路

划分数据集构建二分类模型，对随机森林，XGBoost，深度学习 DNN 三个机器学习模型进行训练、测试同时进行模型之间的对比评估；模型预测目标是：是否为该用户发放商家代金券并使用准确度、分类报告、混淆矩阵来评估模型结果。

4.2 解决方法

随机森林：

本文使用 sklearn 封装的集成模型 RandomForestClassifier 来构建随机森林。

数据预处理：

• 处理缺失值：使用数值 2 填充‘使用状态’列中的缺失值;

• 转换数据类型：将时间戳格式的‘付款日期’转换为日期时间格式，并从中提取出年、月、日和小时作为新特征；

• 编码分类变量：对‘省份’和‘城市’列进行标签编码；

具体实现如代码 16 所示：

代码 16 随机森林数据预处理

```
data['使用状态'].fillna(2, inplace=True)

data['付款日期'] = pd.to_datetime(data['付款日期'])
data['年月'] = data['付款日期'].dt.to_period('M')
data['年'] = data['年月'].dt.year
data['月'] = data['年月'].dt.month
data.drop('年月', axis=1, inplace=True)

data['付款日期'] = pd.to_datetime(data['付款日期'])
data['付款日期'] = data['付款日期'].astype(np.int64)

label_encoder = LabelEncoder()
for column in ['省份', '城市']:
    data[column] = label_encoder.fit_transform(data[column])
```

需要注意的是在原本的数据集类型中：‘年月’的数据类型是 period[M]代表特定的时间周期。

```
订单ID            object
商家ID             int64
用户ID            object
付款日期    datetime64[ns]
使用状态           float64
实付金额             int64
邮费               int64
省份              object
城市              object
购买数量             int64
年月           period[M]
dtype: object
```

但是这样的数据类型无法直接用于模型训练，要将‘年月’ 列转换为适合随机森林模型的格式，需要将其转换为两个单独的列：年和月。这样可以更好地

让模型理解这个特征，同时避免 period[M] 类型带来的问题。
具体实现代码 16 已经给出。

```
订单ID      object
商家ID       int64
用户ID      object
付款日期       int64
使用状态     float64
实付金额       int64
邮费         int64
省份         int32
城市         int32
购买数量       int64
年          int64
月          int64
dtype: object
```

所有的特征都将是数值型，适合用于模型训练。

1. **特征工程：**

选择除‘订单 ID’和‘用户 ID’以外的所有列作为特征，‘使用状态’作为目标变量；具体实现如代码 17 所示。

代码 17 特征工程

```
X = data.drop(['订单ID', '用户ID', '使用状态'], axis=1)
y = data['使用状态']
```

2. 模型构建和训练：

- 划分训练集(80%)和测试集(20%)；
- 使用随机森林分类器进行模型构建；
- 用训练集数据训练模型；

具体实现如代码 18 所示：

代码 18 随机森林模型构建和训练

```
#划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#随机森林
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Model Evaluation
y_pred = model.predict(X_test)
```

XGBoost 模型：

本文使用集成模型库 xgboost 构建 XGBoost 模型，由于 XGBoost 模型和随机森林模型的

数据预处理、特征选择部分是通用的，只描述模型构建与训练部分。

模型构建与训练：

- 划分训练集(80%)和测试集(20%)；
- 使用 XGBoost 模型进行模型构建；
- 用训练集数据训练模型；

具体实现如代码 19 所示：

代码 19　XGBoost 模型构建和训练

```
# XGBoost模型
xgb_model = xgb.XGBClassifier(use_label_encoder=False, eval_metric='mlogloss', n_estimators=100, random_state=42)
xgb_model.fit(X_train, y_train)

# 模型评估
y_pred = xgb_model.predict(X_test)
```

DNN 模型：

本文使用 PyTorch 深度学习框架来构建 DNN 模型，由于 DNN 模型和随机森林模型的

数据预处理、特征选择、划分数据集部分是通用的，所以本文从构建 PyTorch 数据集开始描述。

• 构建 PyTorch 数据集：定义了一个 CouponDataset 类，用于将特征和标签转换为 PyTorch 张量；

• 构建 DNN 模型：定义了一个基于 PyTorch 的简单的深度神经网络，具有三个全连接层；

• 定义损失函数和优化器：使用交叉熵损失函数和 Adam 优化器；

• 模型训练：定义了一个函数 train_model，用于训练模型，包括正向传播、计算损失、反向传播和优化步骤；具体实现如代码 20 所示。

代码 20 DNN 模型

```
# 构建PyTorch Dataset
class CouponDataset(Dataset):
    def __init__(self, features, labels):
        self.features = torch.tensor(features, dtype=torch.float32)
        self.labels = torch.tensor(labels, dtype=torch.long)

    def __len__(self):
        return len(self.features)

    def __getitem__(self, idx):
        return self.features[idx], self.labels[idx]

train_dataset = CouponDataset(X_train, y_train)
test_dataset = CouponDataset(X_test, y_test)

#构建DNN模型
class DNN(nn.Module):
    def __init__(self, input_size, num_classes):
        super(DNN, self).__init__()
        self.fc1 = nn.Linear(input_size, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, num_classes)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

model = DNN(X_train.shape[1], len(np.unique(y)))
```

```python
# 定义损失函数和优化器
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# 训练模型
def train_model(model, train_dataset, criterion, optimizer, epochs=10):
    train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
    for epoch in range(epochs):
        for features, labels in train_loader:
            outputs = model(features)
            loss = criterion(outputs, labels)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

train_model(model, train_dataset, criterion, optimizer, epochs=10)
```

4.3 模型评估

随机森林：

1. 准确度（Accuracy）：

• 模型的准确度为 96.16%。这意味着模型在测试集上正确预测的样本占总样本的 96.16%。

2. 分类报告（Classification Report）：

报告包含了三个类别（0、1 和 2）的精确度（precision）、召回率（recall）和 F1 得分（f1-score）、宏平均（Macro Avg）和加权平均（Weighted Avg）。

• 对于类别 0，精确度为 14%，召回率为 1%，F1 得分为 2%；

• 对于类别 1，精确度为 96%，召回率为 100%，F1 得分为 98%；

• 对于类别 2，精确度为 0%，召回率为 0%，F1 得分为 0%；

• 宏平均考虑了所有类别的精确度、召回率和 F1 得分，每个类别的权重相同，得分分别为 37%，34%，33%；

• 加权平均考虑了每个类别的样本数量，得分分别为 93%，96%，95%，反映了类别 1 在数据集中占主导地位；

这些指标表明，类别 1 的预测效果最好，而类别 0 和 2 的预测效果较差。

3. 混淆矩阵（Confusion Matrix）：

混淆矩阵是一个 3x3 的矩阵，显示了每个类别的真实值和预测值。

类别 0（第一行）：

• 只有 1 个样本被正确分类为类别 0（真阳性）；

• 77 个样本实际上是类别 0，但被错误地分类为类别 1（假阳性）；

• 1 个样本实际上是类别 0，但被错误地分类为类别 2；

类别 1（第二行）：

• 3881 个样本被正确分类为类别 1；

• 6 个样本实际上是类别 1，但被错误地分类为类别 0；

• 3 个样本实际上是类别 1，但被错误地分类为类别 2；

类别 2（第三行）：

• 没有样本被正确分类为类别 2。

• 68 个样本实际上是类别 2，但全部被错误地分类为类别 1。

总体来看，模型在类别 1 上的预测效果非常好，但在类别 0 和 2 上的效果一般。这可能是由于数据不平衡或模型对特定类别的预测能力有限导致的。具体实现如代码 21 所示。

代码 21 随机森林模型评估

```
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

precision = precision_score(y_test, y_pred, average=None)
recall = recall_score(y_test, y_pred, average=None)
f1 = f1_score(y_test, y_pred, average=None)

print(f"Accuracy: {accuracy}")
print(f"Classification Report:\n{report}")
print(f"Confusion Matrix:\n{conf_matrix}")
print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"F1 Score for each class: {f1}")
```

结果 17 评估参数可视化

```
Accuracy: 0.9616051523408472
Classification Report:
              precision    recall  f1-score   support

         0.0       0.14      0.01      0.02        79
         1.0       0.96      1.00      0.98      3890
         2.0       0.00      0.00      0.00        68

    accuracy                           0.96      4037
   macro avg       0.37      0.34      0.33      4037
weighted avg       0.93      0.96      0.95      4037

Confusion Matrix:
[[   1   77    1]
 [   6 3881    3]
 [   0   68    0]]
Precision: [0.14285714 0.9639841  0.        ]
Recall: [0.01265823 0.99768638 0.        ]
F1 Score for each class: [0.02325581 0.98054573 0.        ]
```

XGBoost 模型：

1. 准确度（Accuracy）：

• 模型的准确度为 96.38%。这意味着模型在测试集上正确预测的样本占总样本的 96.38%。

2. 分类报告（Classification Report）：

报告包含了三个类别（0、1 和 2）的精确度（precision）、召回率（recall）和 F1 得分（f1-score）、宏平均（Macro Avg）和加权平均（Weighted Avg）。

• 对于类别 0，精确度为 100%，召回率为 1%，F1 得分为 2%；

• 对于类别 1，精确度为 96%，召回率为 100%，F1 得分为 98%；

• 对于类别 2，精确度为 0%，召回率为 0%，F1 得分为 0%；

• 宏平均考虑了所有类别的精确度、召回率和 F1 得分，每个类别的权重相同，得分分别为 65%，34%，34%；

• 加权平均考虑了每个类别的样本数量，得分分别为 95%，96%，95%，反映了类别 1 在数据集中占主导地位；

这些指标表明，类别 1 的预测效果最好，而类别 0 和 2 的预测效果较差。

3. 混淆矩阵（Confusion Matrix）：

混淆矩阵是一个 3x3 的矩阵，显示了每个类别的真实值和预测值。

类别 0（第一行）：

• 只有 1 个样本被正确分类为类别 0（真阳性）；

• 78 个样本实际上是类别 0，但被错误地分类为类别 1（假阳性）；

• 没有样本被错误地分类为类别 2；

类别 1（第二行）：

• 3890 个样本被正确分类为类别 1。

• 没有样本实际上是类别 1 但被错误地分类为类别 0 或类别 2；

类别 2（第三行）：

• 没有样本被正确分类为类别 2。

• 68 个样本实际上是类别 2，但全部被错误地分类为类别 1。

类别 1 的预测效果极佳，模型成功地将所有类别 1 的样本正确分类。对于类别 0 和类别 2，模型的预测效果非常差。特别是对类别 2，模型完全没能正确分类任何样本。类别 0 和类别 2 的样本几乎全部被误分类为类别 1，这可能是由于数据集中类别 1 的样本数量远远超过其他类别，导致模型在预测时对类别 1 有偏倾向。具体实现如代码 22 所示。

代码 22 XGBoost 模型评估

```
# 模型评估
y_pred = xgb_model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

precision = precision_score(y_test, y_pred, average=None)
recall = recall_score(y_test, y_pred, average=None)
f1 = f1_score(y_test, y_pred, average=None)

print(f"Accuracy: {accuracy}")
print(f"Classification Report:\n{report}")
print(f"Confusion Matrix:\n{conf_matrix}")
print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"F1 Score for each class: {f1}")
```

结果 18 评估参数可视化

```
Accuracy: 0.9638345305920237
Classification Report:
              precision    recall  f1-score   support

         0.0       1.00      0.01      0.02        79
         1.0       0.96      1.00      0.98      3890
         2.0       0.00      0.00      0.00        68

    accuracy                           0.96      4037
   macro avg       0.65      0.34      0.34      4037
weighted avg       0.95      0.96      0.95      4037

Confusion Matrix:
[[   1   78    0]
 [   0 3890    0]
 [   0   68    0]]
```

DNN 模型：

1. 准确度（Accuracy）：
   - 模型的准确度为 96.35%；
2. 分类报告（Classification Report）：
   - 对于类别 0，精确度为 0%，召回率为 0%，F1 得分为 0%；
   - 对于类别 1，精确度为 96%，召回率为 100%，F1 得分为 98%；
   - 对于类别 2，精确度为 0%，召回率为 0%，F1 得分为 0%；
3. 混淆矩阵（Confusion Matrix）：

   类别 0（第一行）：
   - 没有样本被正确分类为类别 0（真阳性）；
   - 79 个样本实际上是类别 0，但被错误地分类为类别 1（假阳性）；
   - 没有样本被错误地分类为类别 2；

   类别 1（第二行）：
   - 3890 个样本被正确分类为类别 1；

• 没有样本实际上是类别 1 但被错误地分类为类别 0 或类别 2；

类别 2（第三行）：

• 没有样本被正确分类为类别 2；

• 68 个样本实际上是类别 2，但全部被错误地分类为类别 1；

表明模型对数据集中的主导类别过于敏感，而对少数类别不够敏感。具体实现如代码 23 所示。

代码 23 DNN 模型评估

```
y_true, y_pred = evaluate_model(model, test_dataset)

# 计算混淆矩阵
cm = confusion_matrix(y_true, y_pred)

# 计算评估指标
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, average='macro')
recall = recall_score(y_true, y_pred, average='macro')
f1 = f1_score(y_true, y_pred, average='macro')
report = classification_report(y_test, y_pred)

# 对于多分类问题，AUC的计算需要将标签二值化
y_true_binarized = label_binarize(y_true, classes=np.unique(y_true))
y_pred_binarized = label_binarize(y_pred, classes=np.unique(y_true))

# 计算每个类的AUC，并计算平均值
n_classes = y_true_binarized.shape[1]
auc = roc_auc_score(y_true_binarized, y_pred_binarized, average='macro', multi_class='ovr')

# 打印评估结果
print(f"Accuracy: {accuracy}")
print(f"Classification Report:\n{report}")
print(f"Confusion Matrix:\n{cm}")
print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"F1 Score for each class: {f1}")
```

结果 19 评估参数可视化

```
Accuracy: 0.9635868218974486
Classification Report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        79
           1       0.96      1.00      0.98      3890
           2       0.00      0.00      0.00        68

    accuracy                           0.96      4037
   macro avg       0.32      0.33      0.33      4037
weighted avg       0.93      0.96      0.95      4037

Confusion Matrix:
[[   0   79    0]
 [   0 3890    0]
 [   0   68    0]]
```

在三个机器学习模型中，类别 1 始终被非常准确地预测，几乎没有误分类的情况。这表明模型在识别类别 1 上非常有效。类别 0 和类别 2 的预测效果普遍不佳。在随机森林模型和 XGBoost 模型中，至少有少量类别 0 的样本被正确分类，但在 DNN 深度学习模型中，类别 0 和类别 2 的样本全部被误分类为类别 1。随着从第一个到第三个模型的变化，观察模型在类别 0 和类别 2 上的预测效果进一步恶化，表明模型对这些类别几乎没有识别能力。

总体来说，模型对于数据集中的主导类别（类别 1）具有很好的识别能力，

但对于少数类别（类别 0 和类别 2）的识别能力非常有限。这可能是由于数据集中类别 1 的样本数量远远多于其他类别，导致模型在预测时对类别 1 有过度的偏倾向。

任务五

5.1 解决思路

基于随机森林模型的预测结果，预测值保留输出概率，这样按照需要优惠券的概率分箱，然后按照梯度给每一箱发放一定比例的优惠券、整体概率越高可以发放比例越高同时保留一定的测试组做 ABtest。

5.2 解决方法

在设计综合的优惠券投放策略时，考虑电商平台的用户行为、订单特征和商家情况。策略的核心目标是提高优惠券的使用率和用户满意度，同时为平台带来更高的回报率。

核心策略概述：

1. 个性化与目标用户群：利用深入的用户行为分析和 RFM 模型，识别不同价值的用户群体，并针对这些群体设计个性化的优惠券。通过机器学习模型预测，我们将更精准地定位可能对优惠券反应积极的用户。

2. 优惠券设计与类型：根据用户的消费模式和偏好，优惠券的类型将涵盖直减、满减和折扣等，旨在满足不同用户的需求。对于频繁小额消费的用户，我们将提供小额折扣或满减优惠，而对于偶尔大额消费的用户，则提供更有吸引力的优惠。

3. 时间与地理因素：考虑用户购买行为的季节性差异和地理位置差异，以设计特定时段和区域的优惠策略，确保优惠券投放的时效性和地域相关性。

4. 商家合作：通过与顶级商家合作，推出专属优惠活动，同时支持新商家进入平台，以增加用户购买意愿和提升商家的市场知名度。

5. 营销渠道多元化：结合社交媒体、邮件营销和应用内推送等多渠道，优化优惠券的推广策略，增加用户接触和参与度。

6. 效果监测与策略调整：持续监控优惠券使用情况和销售数据，并根据数据反馈调整优惠券类型、额度和投放策略，以保持策略的灵活性和有效性。

7. 风险管理：在设计优惠券时，考虑潜在的滥用风险，并通过设定使用条件和期限来进行控制。同时定期评估优惠券对平台利润的影响，确保策略的可持续性。

通过这些策略，电商平台将能更好地吸引和保留客户，提高用户的购买体验，同时提升销售额和市场竞争力。这些策略的实施将是一个持续的过程，需要根据市场反馈和数据分析结果不断进行优化和调整。

参考文献

[1] 王彦霞，基于大数据技术的电商用户数据挖掘探讨[J，商场现代化,2022(23):58-60.

[2] 曹安林，电商数据的挖掘分析及应用[J]，无线互联科技,2022,19(15):97-99.

[3] 陈贤平，基于区块链技术的农村电商信息数据挖掘平台[J]，信息与电脑(理论版),2022,34(9):92-94.

[4] 孙海波，数据挖掘在跨境电商客户特征分析中的应用[J]，电脑知识与技术,2021,17(15):239-241.

[5] 张书月，数据挖掘技术在电商情感规律分析中的应用研究[J]，电脑知识与技术,2021,17(5):258-259.

[6] 谢慧敏，基于数据挖掘的电商企业营销管理研究[J]，品牌研究,2019(11):35-36.

[7] 周志华，机器学习，清华大学出版社：2015：173-178.

[8] 张志坚, 周雅迪, 郭军华, 王鹏. 考虑平台优惠券的不同线上销售决策研究[J]，运筹与管理, 2023, 32(1): 147-153.

[9] Cheng Y S, Li H Y, Thorstenson A. Advance selling with double marketing efforts in a newsvendor framework[J]. Computers & Industrial Engineering, 2018, 118: 352-365.

[10] Cao X G, Wu H, Chen K B, et al. Coordination strategy of the supply chain with consumer preferences under the “presale and spot-sale” mode[J]. Mathematical Problems in Engineering, 2020: 1-15.

附录

源代码

```
import numpy as np

import matplotlib.pyplot as plt

import pandas as pd

import seaborn as sns

from matplotlib.font_manager import FontProperties

from datetime import datetime

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler

from sklearn.ensemble import RandomForestClassifier

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import LabelEncoder

from sklearn.model_selection import train_test_split

from sklearn.metrics import confusion_matrix, accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score, classification_report

import xgboost as xgb

import torch

from torch.utils.data import Dataset, DataLoader

import torch.nn as nn

import torch.nn.functional as F

from sklearn.preprocessing import label_binarize

# 读取数据

file_path = './data/data.xlsx'

data = pd.read_excel(file_path)

print(data[:10])
```

```
data['使用状态'].fillna(2, inplace=True)
data['付款日期'] = pd.to_datetime(data['付款日期'])
data['年月'] = data['付款日期'].dt.to_period('M')
data['实付金额'] = data['实付金额'].abs()
label_encoder = LabelEncoder()
for column in ['省份', '城市']:
    data[column] = label_encoder.fit_transform(data[column])
# 用户维度分析
# 计算每个用户的消费总额
user_total_spending = data.groupby('用户 ID')['实付金额'].sum()
# 计算每个用户的平均订单金额
average_order_amount = data.groupby('用户 ID')['实付金额'].mean()
# 地域维度分析
# 各省份、城市的消费金额和消费数量
province_spending = data.groupby('省份')['实付金额'].sum()
city_spending = data.groupby(['省份', '城市'])['实付金额'].sum()
province_order_count = data.groupby('省份').size()
city_order_count = data.groupby(['省份', '城市']).size()
# 订单维度分析
# 每笔订单的购买数量分布
order_quantity_distribution = data['购买数量'].value_counts()
# 实付金额分布
payment_distribution = data['实付金额'].value_counts().nlargest(100)
# 邮费分布（包邮比例）
free_shipping_ratio = (data['邮费'] == 0).mean()
# 优惠券使用率
coupon_usage_rate = (data['使用状态'] == 1).mean()
```

```
coupon_usage = data['使用状态'].value_counts()
plt.figure(figsize=(10, 10))
coupon_usage.plot(kind='pie', autopct='%1.1f%%', startangle=140)
plt.title('优惠券使用率')
plt.ylabel('')
plt.show()
# 商品维度分析
# 各商家销售情况
merchant_sales = data.groupby('商家 ID')['实付金额'].sum().nlargest(100)
merchant_order_count = data.groupby('商家 ID').size().nlargest(100)
# 可视化示例：用户消费总额分布图
plt.rcParams["font.sans-serif"] = ["SimHei"] #设置字体
plt.rcParams["axes.unicode_minus"] = False #该语句解决图像中的“-”负号的乱码问题
data['付款日期'] = pd.to_datetime(data['付款日期']).dt.date
sales_data = data.groupby('付款日期')['购买数量'].sum().reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_data['付款日期'], sales_data['购买数量'])
plt.title('订单销量随付款日期的变化')
plt.xlabel('付款日期')
plt.ylabel('销量')
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
# #
plt.figure(figsize=(20, 10), dpi=100)
user_total_spending.hist(bins=50)
```

```
plt.title('用户消费总额分布')
plt.xlabel('消费总额')
plt.ylabel('用户数量')
plt.show()
# 用户平均订单金额分布
plt.figure(figsize=(20, 10), dpi=100)
average_order_amount.hist(bins=50)
plt.title('用户平均订单金额分布')
plt.xlabel('平均订单金额')
plt.ylabel('用户数量')
plt.show()

## 省份消费金额分布
plt.figure(figsize=(20, 10), dpi=100)
province_spending.plot(kind='bar')
plt.xticks(rotation=45, fontsize=12)
plt.yticks(fontsize=12)
plt.title('省份消费金额分布')
plt.xlabel('省份')
plt.ylabel('消费金额')
plt.show()

# 省份消费数量分布
plt.figure(figsize=(20, 10), dpi=100)
province_order_count.plot(kind='bar')
plt.title('省份消费数量分布')
plt.xlabel('省份')
```

```
plt.ylabel('订单数量')
plt.xticks(rotation=45)
plt.show()
# 订单维度分析的可视化
#购买数量分布
plt.figure(figsize=(20, 10))
order_quantity_distribution.plot(kind='bar')
plt.title('购买数量分布')
plt.xlabel('购买数量')
plt.ylabel('订单数量')
plt.show()
#
## 实付金额分布
plt.figure(figsize=(20, 10))
payment_distribution.plot(kind='bar')
plt.title('实付金额分布')
plt.xlabel('实付金额')
plt.ylabel('订单数量')
plt.show()
#
## 邮费分布
plt.figure(figsize=(20, 10))
data['邮费'].value_counts().plot(kind='bar')
plt.title('邮费分布')
plt.xlabel('邮费')
plt.ylabel('订单数量')
plt.show()
```

```
#
## 优惠券使用率
coupon_usage = data['使用状态'].value_counts()
plt.figure(figsize=(10, 10))
coupon_usage.plot(kind='pie', autopct='%1.1f%%', startangle=140)
plt.title('优惠券使用率')
plt.ylabel('')
plt.show()
#
# 商品维度分析的可视化
# 商家销售情况
plt.figure(figsize=(10, 6))
merchant_sales.plot(kind='bar')
plt.title('商家销售情况')
plt.xlabel('商家 ID')
plt.ylabel('销售额')
plt.xticks(rotation=90)
plt.show()

# 商品维度分析的可视化
# 商家销售情况
plt.figure(figsize=(20, 10))
merchant_sales.plot(kind='bar')
plt.title('商家销售情况')
plt.xlabel('商家 ID')
plt.ylabel('销售额')
plt.xticks(rotation=90)
```

```
plt.show()
#
## 商家订单数量
plt.figure(figsize=(20, 10))
merchant_order_count.plot(kind='bar')
plt.title('商家订单数量')
plt.xlabel('商家 ID')
plt.ylabel('订单数量')
plt.xticks(rotation=90)
plt.show()

### 箱线图：每个用户的消费金额分布
plt.figure(figsize=(20, 10))
sns.boxplot(x=data['用户 ID'], y=data['实付金额'])
plt.title('每个用户的消费金额分布')
plt.xlabel('') # 隐藏 x 轴标签
plt.ylabel('实付金额')
plt.xticks(rotation=90)  # 如果用户 ID 较多，可以旋转标签以便阅读
plt.show()
#
## 饼图：不同省份的用户数量占比
province_counts = data.drop_duplicates(subset=['用户 ID', '省份'])['省份'].value_counts()
plt.figure(figsize=(10, 6))
province_counts.plot(kind='pie', autopct='%1.1f%%')
plt.title('不同省份的用户数量占比')
plt.ylabel('')  # 隐藏 y 轴标签
plt.show()
```

```
#

## 热图：不同省份和城市的消费金额分布

province_city_spending = data.pivot_table(index='省份', columns='城市', values='实付金额', aggfunc='sum')

plt.figure(figsize=(20, 10))

sns.heatmap(province_city_spending, annot=True, fmt='.0f')

plt.title('不同省份和城市的消费金额分布')

plt.xlabel('城市')

plt.ylabel('省份')

plt.show()

'''

    第二问

'''

# 用户特征

user_features = data.groupby('用户 ID').agg({

    '订单 ID': 'nunique',

    '实付金额': ['sum', 'mean'],

    '购买数量': ['sum', 'mean'],

    '邮费': ['sum', 'mean', lambda x: (x == 0).sum()]

})


user_features.columns = ['订单数', '总金额', '平均订单金额', '购买商品总数', '购买商品平均数', '邮费金额总数', '邮费金额平均数', '包邮订单数']

user_features['优惠券使用数量'] = data.groupby('用户 ID')['使用状态'].apply(lambda x: (x == 1).sum())

user_features['优惠券使用率'] = data.groupby('用户 ID')['使用状态'].apply(lambda x: (x == 1).sum() / len(x))

user_features['邮费占总金额比率'] = user_features['邮费金额总数'] / user_features['总金额']
```

```
pd.set_option('display.max_columns', None)

pd.set_option('display.width', None)



# 商户特征

merchant_features = data.groupby('商家 ID').agg({

    '订单 ID': 'nunique',

    '实付金额': ['sum', 'mean'],

    '购买数量': ['sum', 'mean'],

    '邮费': ['sum', 'mean', lambda x: (x == 0).sum()]

})

merchant_features.columns = ['订单数', '总金额', '平均订单金额', '售出商品总数', '售出商品平均数', '邮费金额总数', '邮费金额平均数', '包邮订单数']

merchant_features['优惠券使用数量'] = data.groupby('商家 ID')['使用状态'].apply(lambda x: (x == 1).sum())

merchant_features['优惠券使用率'] = data.groupby('商家 ID')['使用状态'].apply(lambda x: (x == 1).sum() / len(x))

merchant_features['邮费占总金额比率'] = merchant_features['邮费金额总数'] / merchant_features['总金额']



# 优惠券特征

# coupon_usage = data.groupby(['用户 ID', '使用状态']).size().unstack(fill_value=0)

# coupon_usage['优惠券使用率'] = coupon_usage['已使用'] / (coupon_usage['已使用'] + coupon_usage['领取未使用'] + coupon_usage['未领取'])



# 考虑付款实际，城市

data['付款日期'] = pd.to_datetime(data['付款日期'])
```

```
data['年月'] = data['付款日期'].dt.to_period('M')

merchant_time_analysis = data.groupby(['商家 ID', '年月']).agg({

    '订单 ID': 'nunique',

    '实付金额': ['sum', 'mean'],

    '购买数量': ['sum', 'mean']

}).reset_index()

merchant_time_analysis.columns = ['商家 ID', '年月', '订单数', '总金额', '平均订单金额', '售出商品总数', '售出商品平均数']



province_city_analysis = data.groupby(['省份', '城市', '年月']).agg({

    '订单 ID': 'nunique',

    '实付金额': ['sum', 'mean'],

    '购买数量': ['sum', 'mean']

}).reset_index()

province_city_analysis.columns = ['省份', '城市', '年月', '订单数', '总金额', '平均订单金额', '商品总数', '商品平均数']


'''

    第三问

'''

reference_date = data['付款日期'].max() + pd.Timedelta(days=1)

rfm = data.groupby('用户 ID').agg({

    '付款日期': lambda x: (reference_date - x.max()).days,

    '订单 ID': 'nunique',

    '实付金额': 'sum'

}).reset_index()
```

```
rfm.columns = ['用户 ID', 'Recency', 'Frequency', 'Monetary']


# Ensuring that the RFM metrics are numeric
rfm['Recency'] = pd.to_numeric(rfm['Recency'], errors='coerce')
rfm['Frequency'] = pd.to_numeric(rfm['Frequency'], errors='coerce')
rfm['Monetary'] = pd.to_numeric(rfm['Monetary'], errors='coerce')
#


quantiles = rfm[['Recency', 'Frequency', 'Monetary']].quantile(q=[0.25, 0.5, 0.75])


def rfm_score(x, p, d):
    if x <= d[p][0.25]:
        return 1
    elif x <= d[p][0.50]:
        return 2
    elif x <= d[p][0.75]:
        return 3
    else:
        return 4


rfm['R_Score'] = rfm['Recency'].apply(rfm_score, args=('Recency', quantiles))
rfm['F_Score'] = rfm['Frequency'].apply(rfm_score, args=('Frequency', quantiles))
rfm['M_Score'] = rfm['Monetary'].apply(rfm_score, args=('Monetary', quantiles))
rfm['RFM_Segment'] = rfm['R_Score'].map(str) + rfm['F_Score'].map(str) +
rfm['M_Score'].map(str)
rfm['RFM_Score'] = rfm[['R_Score', 'F_Score', 'M_Score']].sum(axis=1)
```

```
# 识别高价值和低价值客户

high_value_customers = rfm[rfm['RFM_Score'] >= rfm['RFM_Score'].quantile(0.75)]

mid_value_customers = rfm[(rfm['RFM_Score'] > rfm['RFM_Score'].quantile(0.25)) &
(rfm['RFM_Score'] < rfm['RFM_Score'].quantile(0.75))]

low_value_customers = rfm[rfm['RFM_Score'] <= rfm['RFM_Score'].quantile(0.25)]

print("高价值用户：")

print(high_value_customers.head())

print("潜在发展用户：")

print(mid_value_customers.head())

print("需关注用户：")

print(low_value_customers.head())

#

# '''

#        第四问

# '''


data['使用状态'].fillna(2, inplace=True)


data['付款日期'] = pd.to_datetime(data['付款日期'])

data['年月'] = data['付款日期'].dt.to_period('M')

data['年'] = data['年月'].dt.year

data['月'] = data['年月'].dt.month

data.drop('年月', axis=1, inplace=True)


data['付款日期'] = pd.to_datetime(data['付款日期'])

data['付款日期'] = data['付款日期'].astype(np.int64)
```

```
label_encoder = LabelEncoder()

for column in ['省份', '城市']:

    data[column] = label_encoder.fit_transform(data[column])



X = data.drop(['订单 ID', '用户 ID', '使用状态'], axis=1)

y = data['使用状态']


#划分训练集和测试集

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# 随机森林

model = RandomForestClassifier(n_estimators=100, random_state=42)

model.fit(X_train, y_train)


# Model Evaluation

y_pred = model.predict(X_test)


#

# 获取概率预测而不是简单分类

y_pred_proba = model.predict_proba(X_test)[:, 1]  # 获取测试数据中的用户使用优惠券的概率


# 将预测概率与测试集相对应的行合并

test_data = X_test.copy()

test_data['predicted_probability'] = y_pred_proba
```

```
# 将预测概率分箱

bins = np.linspace(0, 1, 11)

test_data['probability_bin'] = pd.cut(test_data['predicted_probability'], bins,
labels=range(10), include_lowest=True)


# 为每个箱子设定优惠券发放比例

coupon_allocation = {i: i * 10 for i in range(10)}


# 分配优惠券的函数

def allocate_coupons(probability_bin, allocation):

    # 处理 NaN 值

    if pd.isna(probability_bin):

        return False

    threshold = allocation[probability_bin]

    return np.random.rand() < threshold / 100


test_data['receive_coupon'] = test_data['probability_bin'].apply(lambda x:
allocate_coupons(x, coupon_allocation))


# 打印分析报告

print("优惠券发放策略：")

print(test_data[['probability_bin', 'receive_coupon']])




# XGBoost 模型
```

```
xgb_model = xgb.XGBClassifier(use_label_encoder=False, eval_metric='mlogloss',
n_estimators=100, random_state=42)

xgb_model.fit(X_train, y_train)


# 模型评估

y_pred = xgb_model.predict(X_test)


accuracy = accuracy_score(y_test, y_pred)

report = classification_report(y_test, y_pred)

conf_matrix = confusion_matrix(y_test, y_pred)


precision = precision_score(y_test, y_pred, average=None)

recall = recall_score(y_test, y_pred, average=None)

f1 = f1_score(y_test, y_pred, average=None)


print(f"Accuracy: {accuracy}")

print(f"Classification Report:\n{report}")

print(f"Confusion Matrix:\n{conf_matrix}")

print(f"Precision: {precision}")

print(f"Recall: {recall}")

print(f"F1 Score for each class: {f1}")
```

DNN 模型

```
data['使用状态'].fillna(2, inplace=True)

data['年'] = data['付款日期'].dt.year
```

```
data['月'] = data['付款日期'].dt.month

data.drop(['年月'], axis=1, inplace=True, errors='ignore')

data['付款日期'] = pd.to_datetime(data['付款日期']).view(np.int64)


# 标签编码

label_encoder = LabelEncoder()

for column in ['省份', '城市']:

    data[column] = label_encoder.fit_transform(data[column])


# 特征和目标变量

X = data.drop(['订单 ID', '用户 ID', '使用状态'], axis=1)

y = label_encoder.fit_transform(data['使用状态'])


# 数据标准化

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)


# 划分数据集

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)


#构建 PyTorch Dataset

class CouponDataset(Dataset):

    def __init__(self, features, labels):

        self.features = torch.tensor(features, dtype=torch.float32)

        self.labels = torch.tensor(labels, dtype=torch.long)
```

```
    def __len__(self):
        return len(self.features)

    def __getitem__(self, idx):
        return self.features[idx], self.labels[idx]

train_dataset = CouponDataset(X_train, y_train)
test_dataset = CouponDataset(X_test, y_test)

#构建 DNN 模型
class DNN(nn.Module):
    def __init__(self, input_size, num_classes):
        super(DNN, self).__init__()
        self.fc1 = nn.Linear(input_size, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, num_classes)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

model = DNN(X_train.shape[1], len(np.unique(y)))

# 定义损失函数和优化器
criterion = nn.CrossEntropyLoss()
```

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)


# 训练模型
def train_model(model, train_dataset, criterion, optimizer, epochs=10):
    train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
    for epoch in range(epochs):
        for features, labels in train_loader:
            outputs = model(features)
            loss = criterion(outputs, labels)


            optimizer.zero_grad()
            loss.backward()
            optimizer.step()


train_model(model, train_dataset, criterion, optimizer, epochs=10)


# 评估模型
def evaluate_model(model, test_dataset):
    test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
    y_true = []
    y_pred = []
    with torch.no_grad():
        for features, labels in test_loader:
            outputs = model(features)
            _, predicted = torch.max(outputs.data, 1)
            y_pred.extend(predicted.numpy())
            y_true.extend(labels.numpy())
```

```
    return y_true, y_pred


y_true, y_pred = evaluate_model(model, test_dataset)


# 计算混淆矩阵
cm = confusion_matrix(y_true, y_pred)


# 计算评估指标
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, average='macro')
recall = recall_score(y_true, y_pred, average='macro')
f1 = f1_score(y_true, y_pred, average='macro')
report = classification_report(y_test, y_pred)


# 对于多分类问题，AUC 的计算需要将标签二值化
y_true_binarized = label_binarize(y_true, classes=np.unique(y_true))
y_pred_binarized = label_binarize(y_pred, classes=np.unique(y_true))


# 计算每个类的 AUC，并计算平均值
n_classes = y_true_binarized.shape[1]
auc = roc_auc_score(y_true_binarized, y_pred_binarized, average='macro',
multi_class='ovr')


# 打印评估结果
print(f"Accuracy: {accuracy}")
print(f"Classification Report:\n{report}")
print(f"Confusion Matrix:\n{cm}")
```

```
print(f"Precision: {precision}")

print(f"Recall: {recall}")

print(f"F1 Score for each class: {f1}")
```